

Project 1: California Housing Prices

Introduction

This report follows the steps and instructions provided through Chapter 2 of *Hands-On Machine Learning with Scikit-learn, Keras & TensorFlow*. Almost all of the code has been provided from this chapter and many of the discussion points are presented in the text. This report analyzes what has been discussed in the text and takes the time to summarize these thoughts. The logic and reasoning for certain decisions for the project will be illustrated throughout this paper. Here is the general outline of our investigation:

1. Frame the Problem and Look at the Big Picture
2. Get the Data
3. Explore the Data
4. Prepare the Data
5. Shortlist of Promising Models
6. Fine-Tune the System
7. Present Your Solution
8. Summary

Frame the Problem and Look at the Big Picture

We will use California census data to build a model of housing prices in the state. It should be noted that this data is from the 1990 census, but it is still workable for this project. Our model should be able to predict the median housing price in any district. The model's output will be fed to another machine learning system, along with other signals. This downstream system will determine whether it's worth investing in a given area.

The current solution to the problem is that district housing prices are estimated manually by experts. A team gathers up-to-date information about a district, and when they cannot get the median housing price, they estimate it using complex rules. This is costly, time-consuming, and estimates are not great (estimates are off by more than 30%).

The model will employ supervised learning, given the data is labeled. It will most likely be a multiple regression problem because there are multiple features to make the prediction. The model is intended to have a single output, meaning it will be a univariate regression as well. This model will employ batch (offline) learning since there is no continuous flow of data into the system, the data will not be changing rapidly or frequently, and the data will be small enough to fit in memory.

The performance measurement that will be used on this model will be the root mean square error (RMSE, aka Euclidean norm). This is a typical performance measure for regression models, but it should be noted that it is particularly sensitive to outliers. Minimizing RMSE aligns with our business objective of providing accurate housing price predictions.

List of Current Assumptions:

1. The output of the model will be used as a numerical price.
2. The census data provided is comprehensive and contains all the necessary features required for an accurate prediction.
3. An RMSE of less than 50,000 would be considered acceptable for this project.
4. Human expertise is available to validate and provide insights into the model's predictions.

Our next step is to get the data.

Get the Data

For this project, we will be working in a Jupyter notebook in Google Colab. The language will be *Python*, and we will be working with many libraries. Primary libraries: **sklearn**, **pandas**, **numpy**, **scipy**, **matplotlib**, **zlib**.

The data is available in the open-source book repository as a .tgz file:

<https://github.com/ageron/data/raw/main/housing.tgz>

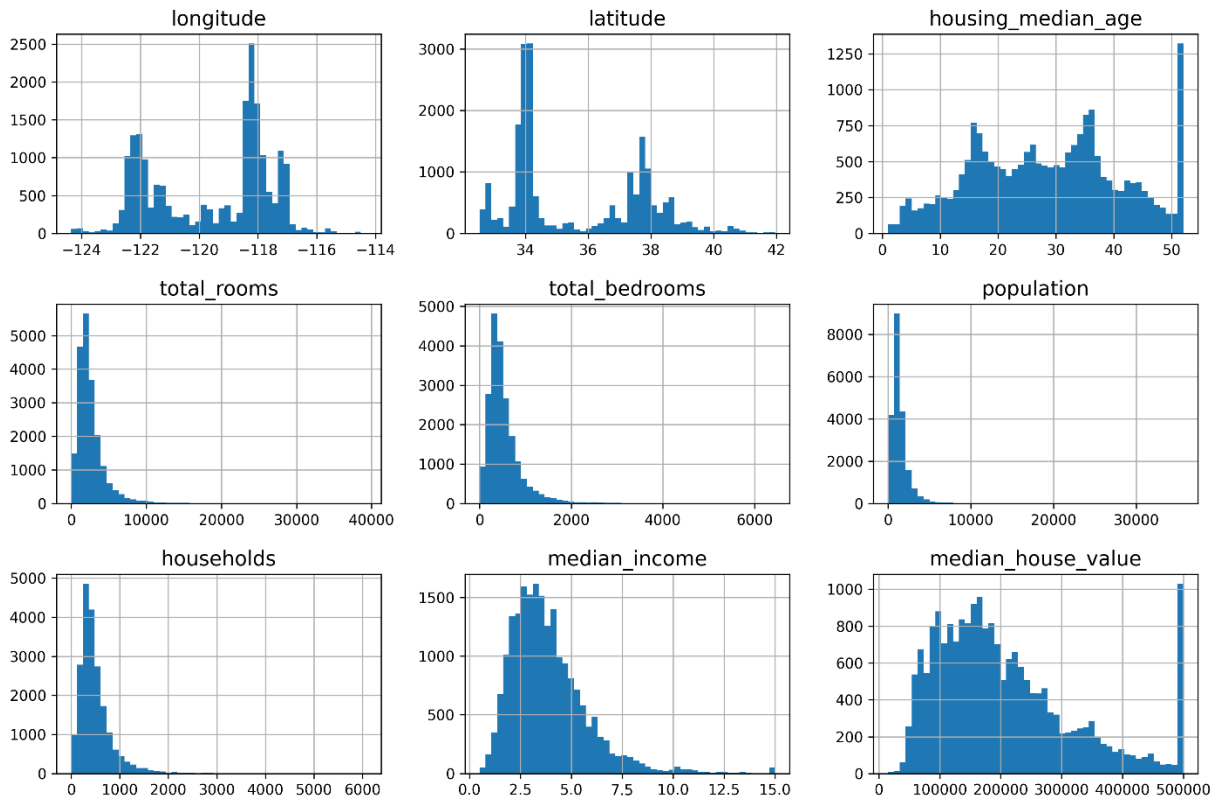
This file will be extracted into a workable, .csv file. This will happen automatically via code.

With the data in the project, we run some code to take a quick look at the features and get a sense of the data. We see if the data is numerical, categorical, text, etc.

```
housing.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype  
---  -
 0   longitude            20640 non-null  float64
 1   latitude             20640 non-null  float64
 2   housing_median_age   20640 non-null  float64
 3   total_rooms          20640 non-null  float64
 4   total_bedrooms       20433 non-null  float64
 5   population           20640 non-null  float64
 6   households           20640 non-null  float64
 7   median_income        20640 non-null  float64
 8   median_house_value   20640 non-null  float64
 9   ocean_proximity      20640 non-null  object  
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```

We are able to visualize the features in histograms.



Somethings to note:

1. Median income attribute is not in US dollars. It has been scaled and capped between 15 and 0.5 and each value is roughly in the ten thousands.
2. Housing median age and median housing value are also capped.
3. The attributes have different scales.
4. Many of the histograms are skewed right.

We will generate our test data during this phase. It is very important that we do not look into the test data when it is generated, also known as data snooping. Looking at the test data has a high tendency to create a bias or a desire to do some cleaning which has a higher likelihood of overfitting the data.

A solution to generating test data is selected after exploring various methods. We will go with a model available through **sklearn** library: `train_test_split()`.

Python:

```
strat_train_set, strat_test_set = train_test_split(
    housing, test_size=0.2, stratify=housing["income_cat"], random_state=42)
```

Let's breakdown how this will work:

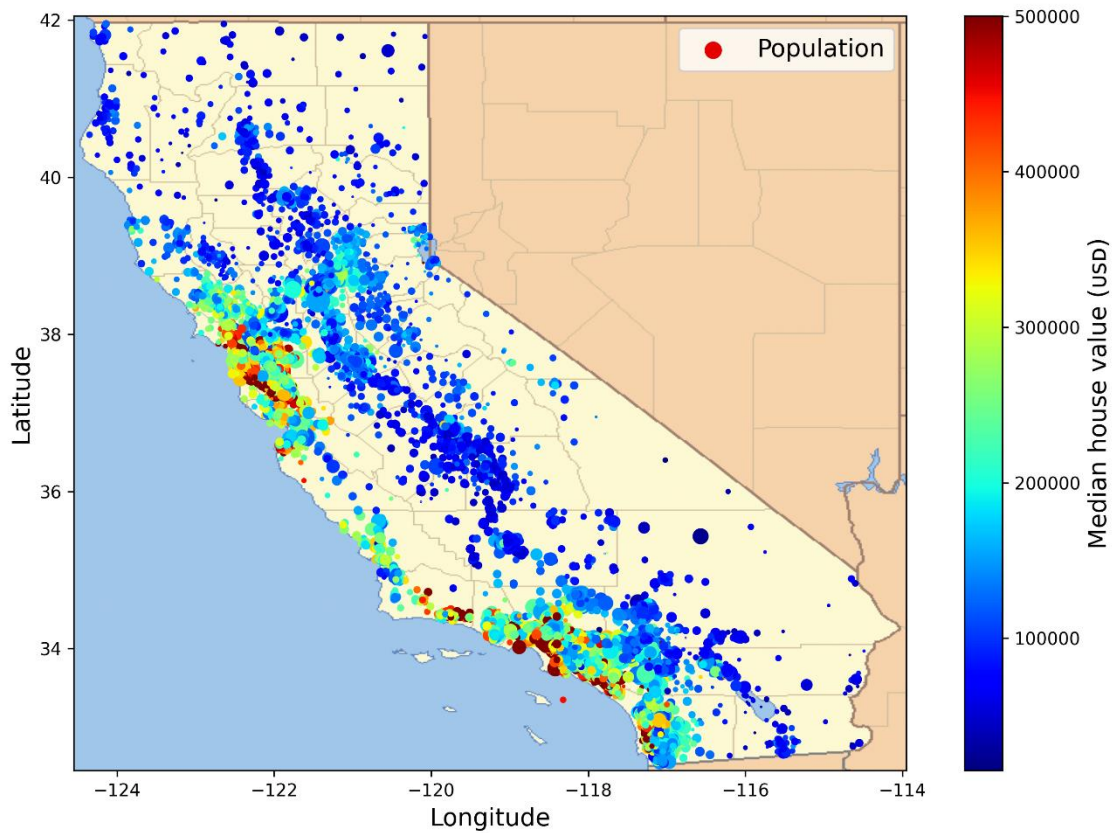
1. The function splits the data into a *strat_train_set* and *strat_test_set*
2. *housing* is the dataframe being used.
3. *test_size=0.2* creates a test set at 20% of the data and a training set at 80%
4. *stratify=housing["income_cat"]* uses the column *income_cat* to ensure the split maintains the same distribution of this variable in both the training and test sets.
5. *random_state=42* sets a seed to generate consistent results every time the code is ran.

With our data in hand, we will now take a deeper dive into the inner workings.

Explore the Data

As previously discussed, the primary purpose of the business solution is to be able to predict the median housing price in any district.

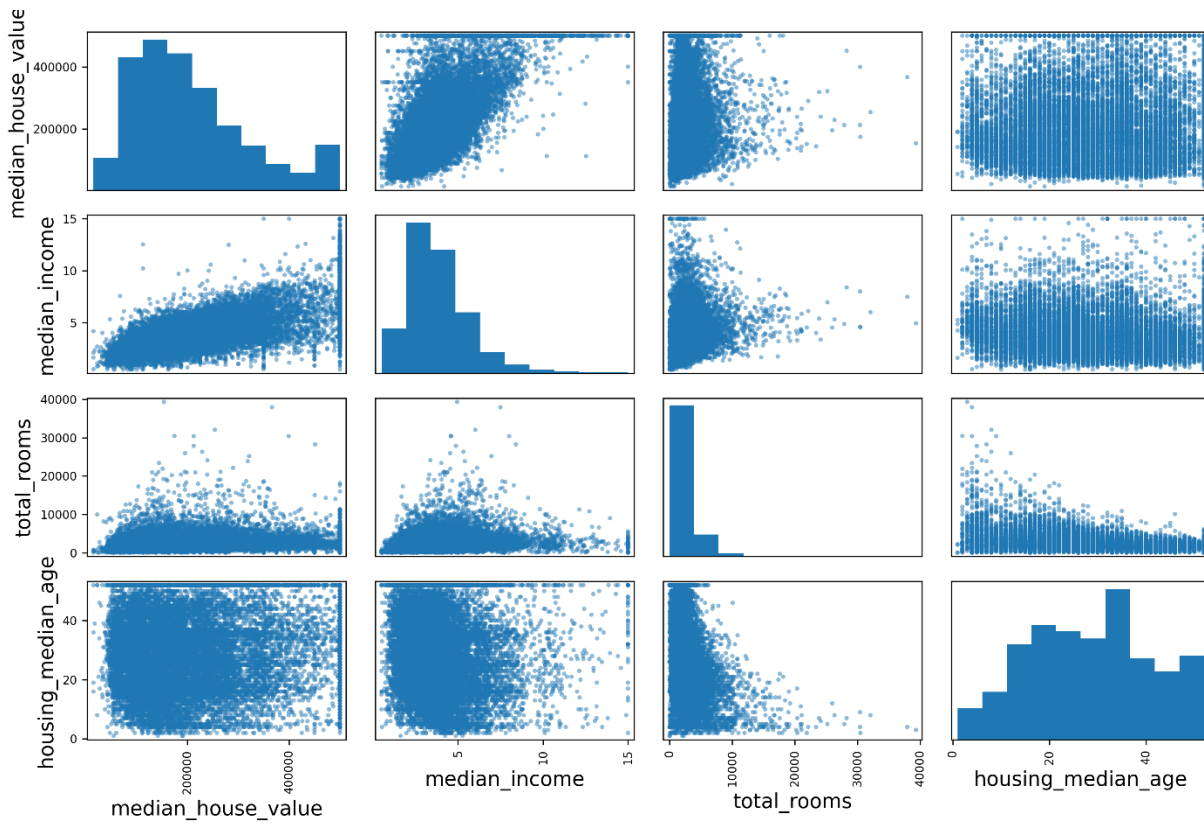
We can use a heat map to visualize the dispersion of data. This visual aid can improve the ability to conceptualize the data.



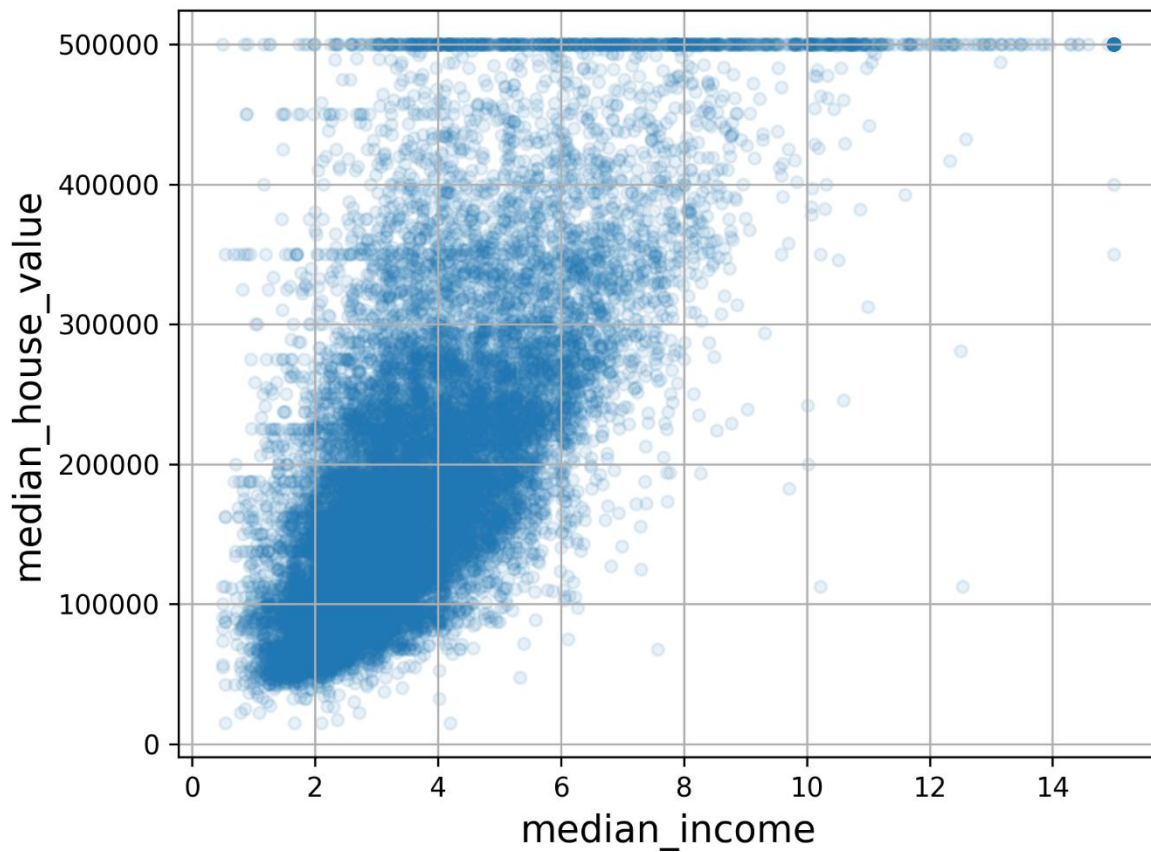
We can also analyze correlations between the data. More specifically, we can analyze the data attributes as they relate to *median_house_value*. In our correlation matrix, we can see the *median_income* has a strong positive correlation with *median_house_value* at 0.688.

	median_house_value
median_house_value	1.000000
median_income	0.688380
total_rooms	0.137455
housing_median_age	0.102175
households	0.071426
total_bedrooms	0.054635
population	-0.020153
longitude	-0.050859
latitude	-0.139584
dtype: float64	

We can also use a scatter matrix as another method to visualize and validate correlations between data.



We can visually see a strong correlation between *median_house_value* and *median_income*. We will isolate the plot.



With a deeper understanding of the data, we can now begin preparing the data for our ML model.

Prepare the Data

The raw data is not ready for use. There needs to be some data cleanup and some transforming of the data to make it useable.

The data has some missing values. There are multiple ways to handle this: remove rows, remove columns, impute the missing values. We will choose to impute the missing values as it does the best at maintaining the data integrity.

The attribute *total_bedrooms* appears to have missing values. The values will be imputed by finding the median value of the column.

There were also significant outliers identified when exploring the data. The Jupyter notebook associated with the book provided a function to remove outliers. We have opted to clean outliers to make our RMSE more reliable when evaluating performance.

There is a column in the dataframe that is categorical, which will require some additional steps. We are able to use **sklearn** method *OneHotEncoder* to generate a useable binary matrix based on our categorical attribute.

This data will be standardized to make the data more workable with the model we select.

There is a Custom Transformer that has been created called *ClusterSimilarity* that is able to perform k-means clustering on selected data.

All of these various techniques are incorporated into multiple steps of a pipeline for the selected model to use. This will allow for the data to be cleaned and transformed in a manner that will be useful for our model.

1. *ratio_pipeline*
2. *log_pipeline*
3. *cluster_pipeline*
4. *cat_pipeline*
5. *default_num_pipeline*

```
[118] def column_ratio(X):
    return X[:, [0]] / X[:, [1]]

def ratio_name(function_transformer, feature_names_in):
    return ["ratio"] # feature names out

def ratio_pipeline():
    return make_pipeline(
        SimpleImputer(strategy="median"),
        FunctionTransformer(column_ratio, feature_names_out=ratio_name),
        StandardScaler())

log_pipeline = make_pipeline(
    SimpleImputer(strategy="median"),
    FunctionTransformer(np.log, feature_names_out="one-to-one"),
    StandardScaler())

cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42)
default_num_pipeline = make_pipeline(SimpleImputer(strategy="median"),
                                     StandardScaler())

preprocessing = ColumnTransformer([
    ("bedrooms", ratio_pipeline(), ["total_bedrooms", "total_rooms"]),
    ("rooms_per_house", ratio_pipeline(), ["total_rooms", "households"]),
    ("people_per_house", ratio_pipeline(), ["population", "households"]),
    ("log", log_pipeline, ["total_bedrooms", "total_rooms", "population",
                           "households", "median_income"]),
    ("geo", cluster_simil, ["latitude", "longitude"]),
    ("cat", cat_pipeline, make_column_selector(dtype_include=object)),
],
    remainder=default_num_pipeline) # one column remaining: housing_median_age
```

Our data is now ready to be used.

Shortlist Promising Models

Earlier we decided that this problem would benefit from a regression model. We explored *LinearRegression*, *DecisionTreeRegressor*, and *RandomForestRegressor*.

We applied **sklearn** k_-fold cross-validation to find the RMSE and see how well each model performed on the data. Our results are slightly different from the book because we opted to remove outliers, but the trend is the same.

1. *LinearRegression* RMSE: 67856.97
2. *DecisionTreeRegressor* RMSE: 65575.96
3. *RandomForestRegressor* RMSE: 45712.81

Evaluating these models, we have selected *RandomForestRegressor* due to its significantly lower RMSE, indicating better predictive performance. The *RandomForestRegressor*'s ability to handle overfitting and provide more stable predictions makes it a solid choice for our dataset.

With the model selected, it is now time to refine the system.

Fine-Tune the System

We can manually fine tune the hyperparameters for our models, but there are built in automations to speed up the process. We explored *GridSearchCV* and *RandomizedSearchCV*. There is a preference for *RandomizedSearchCV* especially when hyperparameter space is large.

We are able to run this tool to be able to identify what are the best hyperparameters for our model. Here is an example table that we can look at.

	<code>n_clusters</code>	<code>max_features</code>	<code>split0</code>	<code>split1</code>	<code>split2</code>	<code>mean_test_rmse</code>
1	45	9	39937	42053	40021	40670
8	32	7	40366	42252	40730	41116
5	42	4	40581	42688	40947	41405
0	41	16	40695	42780	41005	41493
2	23	8	41073	42910	41655	41879

Setting the `n_clusters` to 45 and `max_features` to 9 in our model provides us with the lowest RMSE.

We could look at implementing an ensemble method to further refine the model. One example is to take a look at the *feature_importance* of each attribute and consider removing low significance features.

The last step in the fine-tuning process is to use our test data on our model and evaluate its performance. This can be an iterative process until we have locked down a model, hyperparameter settings, and processed data that meets our needs.

We create a dataframe that drops the feature to be tested (*median_house_value*) and run predictions. We then find the RMSE of the complete test data compared to the new predictions. We can then take the RMSE and run some statistical analysis to see if the value is reasonable or if it is off.

```
X_test = strat_test_set.drop("median_house_value", axis=1)
y_test = strat_test_set["median_house_value"].copy()

final_predictions = final_model.predict(X_test)

#final_rmse = mean_squared_error(y_test, final_predictions, squared=False)
final_rmse = root_mean_squared_error(y_test, final_predictions)
print(final_rmse)

42537.741264526725
```

In our case, our RMSE is 42537.74 on a confidence interval based on t-score of [40383.78, 44587.77] with 95% confidence that the value resides in that interval. We can be confident that your model is performing as intended and the output of the model is reliable.

With our model finalized, it is time to present our solution.

Present Your Solution

Our project focused on predicting housing prices in California using the 1990 census data. We started by acquiring and exploring the data, identifying key features and correlations. Test data is also generated. Cleaning the data involved filling in missing values, removing outliers, and standardizing it. Different models were tested, including *LinearRegression*, *DecisionTreeRegressor*, and *RandomForestRegressor*. Among them, *RandomForestRegressor* performed the best. Fine-tuning it with *RandomizedSearchCV* helped achieve a reliable RMSE.

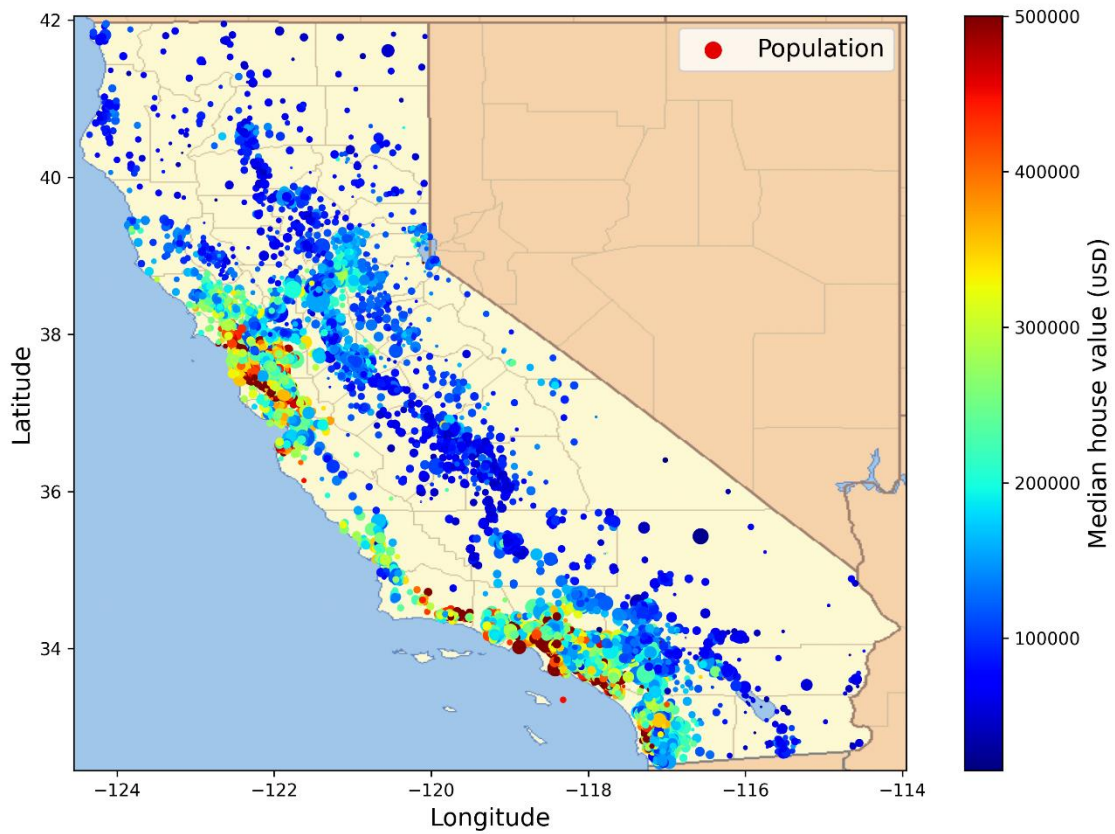
Overview:

Here is a glimpse of some details and the features of the data:

```
housing.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype  
---  -
 0   longitude           20640 non-null  float64
 1   latitude            20640 non-null  float64
 2   housing_median_age  20640 non-null  float64
 3   total_rooms         20640 non-null  float64
 4   total_bedrooms      20433 non-null  float64
 5   population          20640 non-null  float64
 6   households          20640 non-null  float64
 7   median_income       20640 non-null  float64
 8   median_house_value  20640 non-null  float64
 9   ocean_proximity     20640 non-null  object  
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```

With the use of visual aids and tables, we are able to develop a better sense of the data that we are working with.



median_house_value	
median_house_value	1.000000
median_income	0.688380
total_rooms	0.137455
housing_median_age	0.102175
households	0.071426
total_bedrooms	0.054635
population	-0.020153
longitude	-0.050859
latitude	-0.139584
dtype: float64	

From analyzing the data, we can begin preparing the data for machine learning. We have the task of cleaning up missing data and outliers, while also transforming some data to be useable. The use of pipelines are used to prepare the data for use by the models.

```
[118] def column_ratio(X):
    return X[:, [0]] / X[:, [1]]

def ratio_name(function_transformer, feature_names_in):
    return ["ratio"] # feature names out

def ratio_pipeline():
    return make_pipeline(
        SimpleImputer(strategy="median"),
        FunctionTransformer(column_ratio, feature_names_out=ratio_name),
        StandardScaler())

log_pipeline = make_pipeline(
    SimpleImputer(strategy="median"),
    FunctionTransformer(np.log, feature_names_out="one-to-one"),
    StandardScaler())

cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42)
default_num_pipeline = make_pipeline(SimpleImputer(strategy="median"),
                                     StandardScaler())

preprocessing = ColumnTransformer([
    ("bedrooms", ratio_pipeline(), ["total_bedrooms", "total_rooms"]),
    ("rooms_per_house", ratio_pipeline(), ["total_rooms", "households"]),
    ("people_per_house", ratio_pipeline(), ["population", "households"]),
    ("log", log_pipeline, ["total_bedrooms", "total_rooms", "population",
                          "households", "median_income"]),
    ("geo", cluster_simil, ["latitude", "longitude"]),
    ("cat", cat_pipeline, make_column_selector(dtype_include=object)),
],
    remainder=default_num_pipeline) # one column remaining: housing_median_age
```

With the data prepared, we were able to explore a handful of models. We had determined that this was a regression problem and such, explored regression models. It was ultimately determined that the *RandomForestRegressor* model provided us with the best results.

1. *LinearRegression* RMSE: 67856.97
2. *DecisionTreeRegressor* RMSE: 65575.96
3. *RandomForestRegressor* RMSE: 45712.81

```
[131] from sklearn.ensemble import RandomForestRegressor

forest_reg = make_pipeline(preprocessing,
                             RandomForestRegressor(random_state=42))
forest_rmse = -cross_val_score(forest_reg, housing, housing_labels,
                               scoring="neg_root_mean_squared_error", cv=10)
```

```
pd.Series(forest_rmse).describe()
```

	0
count	10.000000
mean	45712.814157
std	1526.379944
min	43581.732899
25%	44586.595164
50%	45444.484552
75%	47069.509995
max	47938.112716

dtype: float64



With the model selected, we take our time to fine-tune the model. We evaluate where improvements can be made in the system. There are functions that allow us to automate the task of evaluating optimal values for hyperparameters.

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

param_distributions = {'preprocessing__geo__n_clusters': randint(low=3, high=50),
                       'random_forest__max_features': randint(low=2, high=20)}

rnd_search = RandomizedSearchCV(
    full_pipeline, param_distributions=param_distributions, n_iter=10, cv=3,
    scoring='neg_root_mean_squared_error', random_state=42)

rnd_search.fit(housing, housing_labels)
```

	n_clusters	max_features	split0	split1	split2	mean_test_rmse
1	45	9	39937	42053	40021	40670
8	32	7	40366	42252	40730	41116
5	42	4	40581	42688	40947	41405
0	41	16	40695	42780	41005	41493
2	23	8	41073	42910	41655	41879

Lastly, we test our model against our test data. With an RMSE of 42537.74 on a confidence interval based on t-score of [40383.78, 44587.77] with 95% confidence that the value resides in that interval

```
X_test = strat_test_set.drop("median_house_value", axis=1)
y_test = strat_test_set["median_house_value"].copy()

final_predictions = final_model.predict(X_test)

#final_rmse = mean_squared_error(y_test, final_predictions, squared=False)
final_rmse = root_mean_squared_error(y_test, final_predictions)
print(final_rmse)

42537.741264526725
```

Our model meets the business objective and is ready to deploy.

Summary

This project was very insightful. I took my time understanding the discussions throughout the example. The progression of the project was logical and easy to follow. The flow felt intuitive. I found myself revisiting the material multiple times for some concepts to make sense. I appreciated that the author presented multiple approaches and solutions in each subsection. The thought and rationale behind each solution, and why one might be better than another, were very informative.

Working with the code in the Jupyter notebook was invaluable. The book material alone might have been overwhelming and less impactful. On the other hand, the Jupyter notebook without the book's context would be harder to follow. Having various solutions enhanced my overall comprehension of the task at hand.

This project also reinforced many terminologies and concepts from Chapter 1. Working through the guided project and applying these terms and concepts made the material more approachable.

Overall, this was a very good experience. It was challenging, with a steep learning curve. There were many "ah ha" moments as I reflected on my calculus, linear algebra, and statistics courses. I now feel more confident approaching the material and look forward to learning more.